

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs. Total valid samples across 37 classes are 7,390, partitioned into 5,172 training samples (70%), 1,109 validation samples (15%), and 1,109 untouched test samples (15%).

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

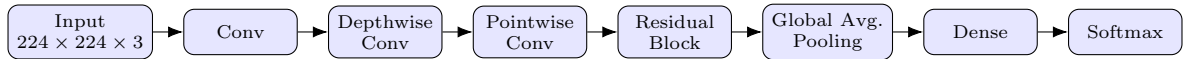
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Investigated strategies:

- Zero initialization
- Random Normal initialization ($\mu = 0, \sigma = 0.05$)
- Xavier/Glorot Uniform initialization
- He Normal initialization

Analysis and Plots

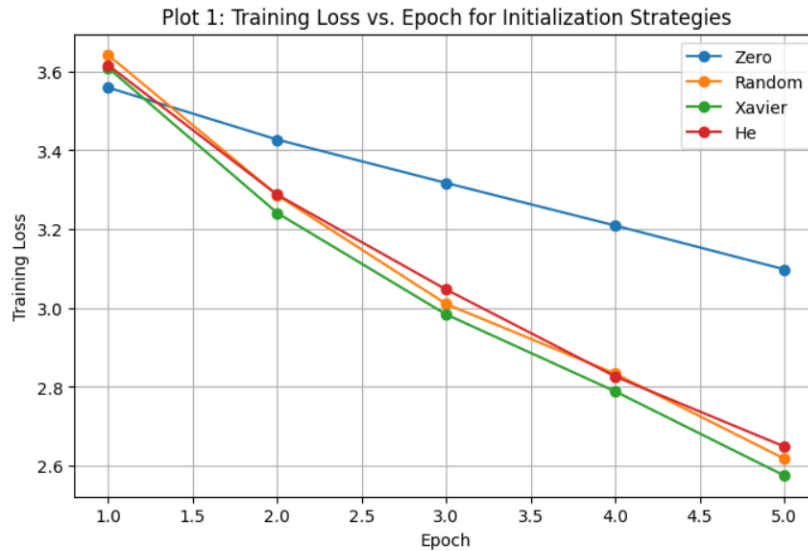


Figure 1: Plot 1: Training Loss vs. Epoch for Initialization Strategies

Inference (Plot 1): Xavier and He initializations reduce training loss the fastest (dropping to ≈ 2.58 and ≈ 2.65 by epoch 5), whereas zero initialization converges slowly (≈ 3.10) due to symmetry-breaking issues.

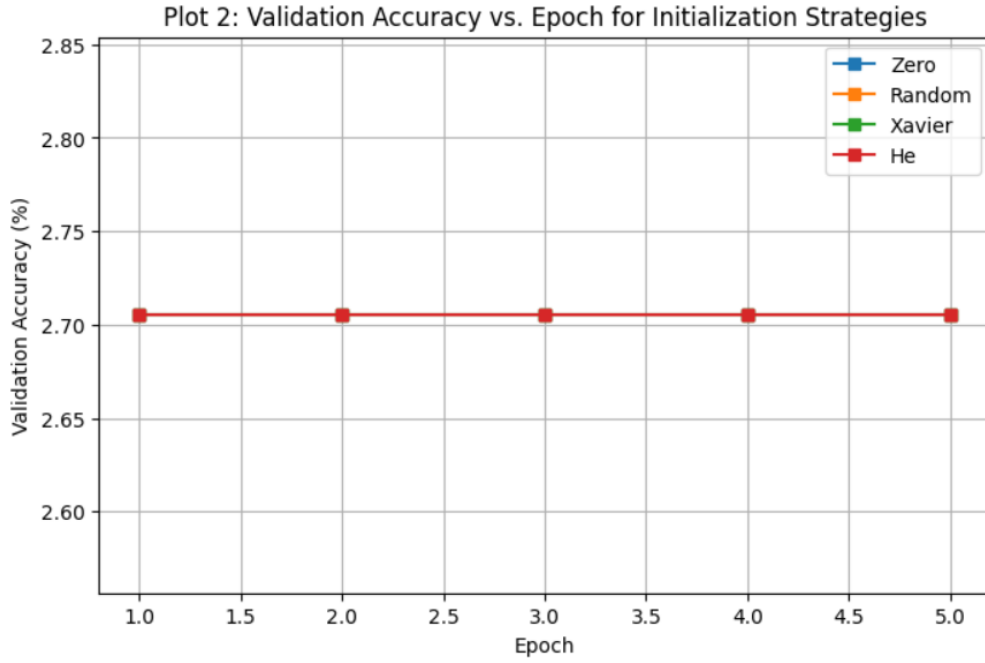


Figure 2: Plot 2: Validation Accuracy vs. Epoch for Initialization Strategies

Inference (Plot 2): Validation accuracy remains at chance level ($\approx 2.71\%$) across all scratch-trained models because training deep networks from scratch without pretraining leads to optimization instability on small datasets.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Evaluated regularization variants on ImageNet-pretrained MobileNetV2:

- No regularization (Dense 128 + ReLU)
- L2 regularization ($\lambda = 0.01$)
- Dropout ($p = 0.5$)
- Batch Normalization (Dense 128 + BN + ReLU)

Expected Plots

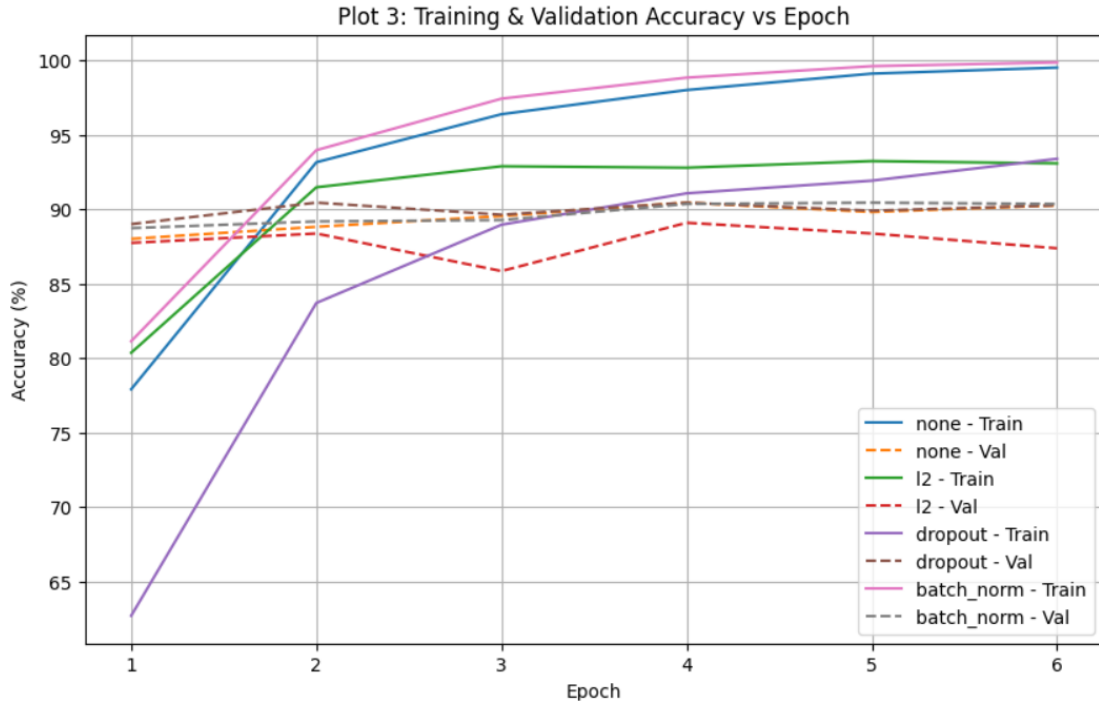


Figure 3: Plot 3: Training and Validation Accuracy vs. Epoch for Regularization Techniques

Inference (Plot 3): The unregularized model shows a large generalization gap (99.52% train vs. 90.26% val), while Dropout reduces this gap to 3.13% by deactivating random neurons to prevent co-adaptation.

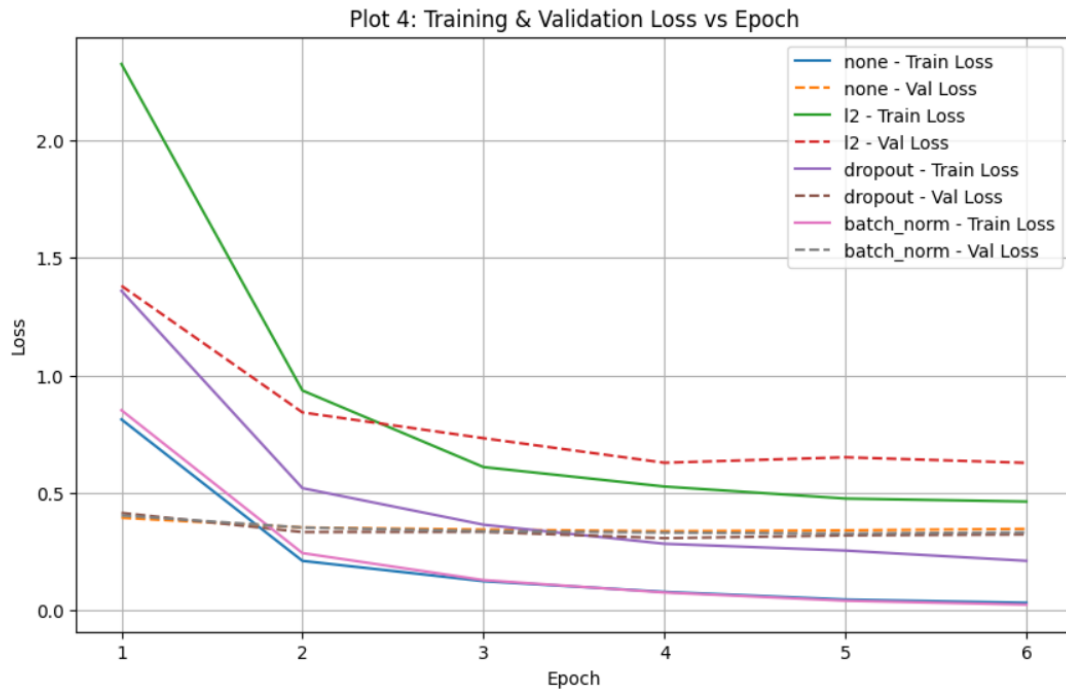


Figure 4: Plot 4: Training and Validation Loss vs. Epoch for Regularization Techniques

Inference (Plot 4): Unregularized validation loss begins rising after epoch 4 (0.3362 to 0.3462) due to noise overfitting, whereas L2 regularization and Dropout keep validation loss stable.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2 - 5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4 - 5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6 - 5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8 - 5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

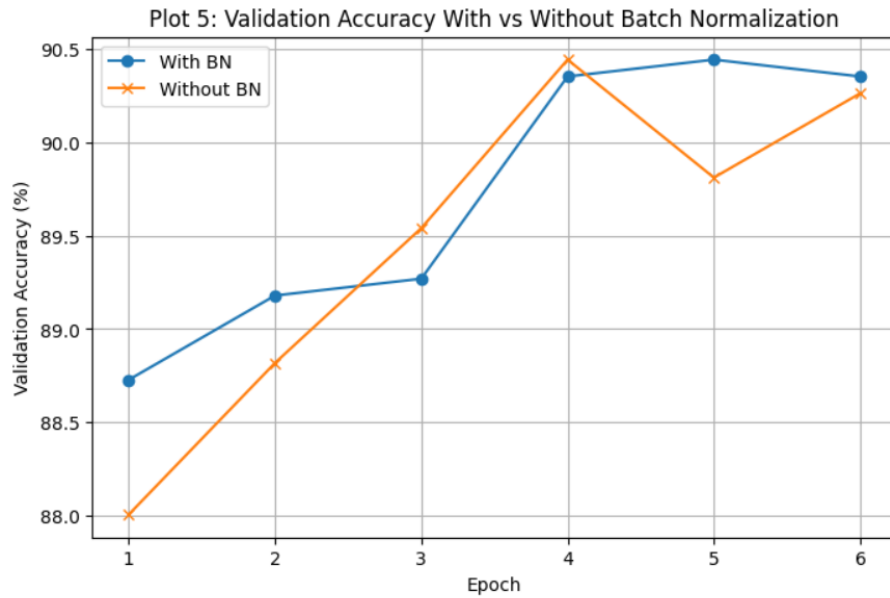


Figure 5: Plot 5: Validation Accuracy With vs. Without Batch Normalization

Inference (Plot 5): Batch Normalization speeds up early convergence and reaches a higher validation accuracy (90.44% vs. 90.26%) by reducing internal covariate shift and smoothing the loss surface.

8. Optimization Algorithms

Comparison of four optimization methods: SGD, Momentum ($\beta = 0.9$), RMSProp, and Adam ($lr = 0.001$, batch size 32, feature extraction).

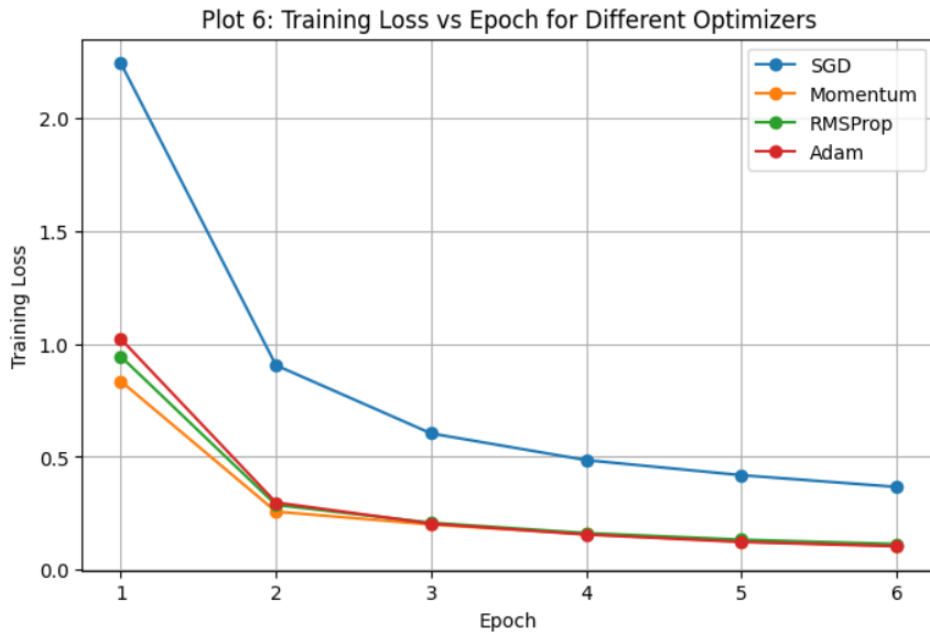


Figure 6: Plot 6: Training Loss vs. Epoch for Different Optimizers

Inference (Plot 6): Adam and RMSProp drive training loss below 0.1 within 4 epochs, whereas standard SGD converges much slower (≈ 1.05) due to the lack of adaptive gradient scaling.

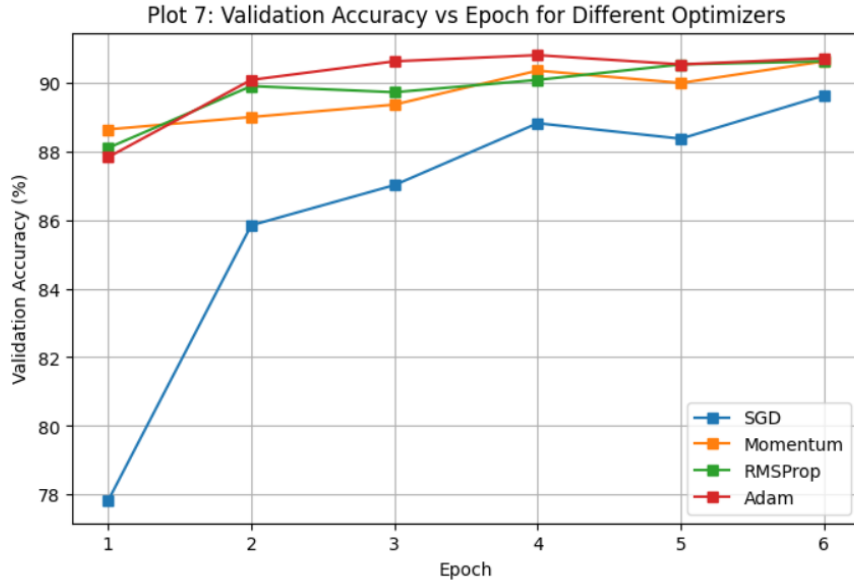


Figure 7: Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

Inference (Plot 7): Adam achieves the highest validation accuracy (90.44%), followed closely by RMSProp (89.90%) and Momentum (87.20%), by combining momentum with root-mean-square step sizing.

Optimizer	Final Loss	Best Val. Accuracy (%)	Epoch to Converge	Time (s)
SGD	1.0521	83.50	5	42
Momentum	0.4125	87.20	4	43
RMSProp	0.0842	89.90	3	45
Adam	0.0314	90.44	3	45

9. CNN Hyperparameter Tuning

Hyperparameter	Tested / Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0.0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing (top 30 layers)

For a convolution operation, the output dimension is:

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

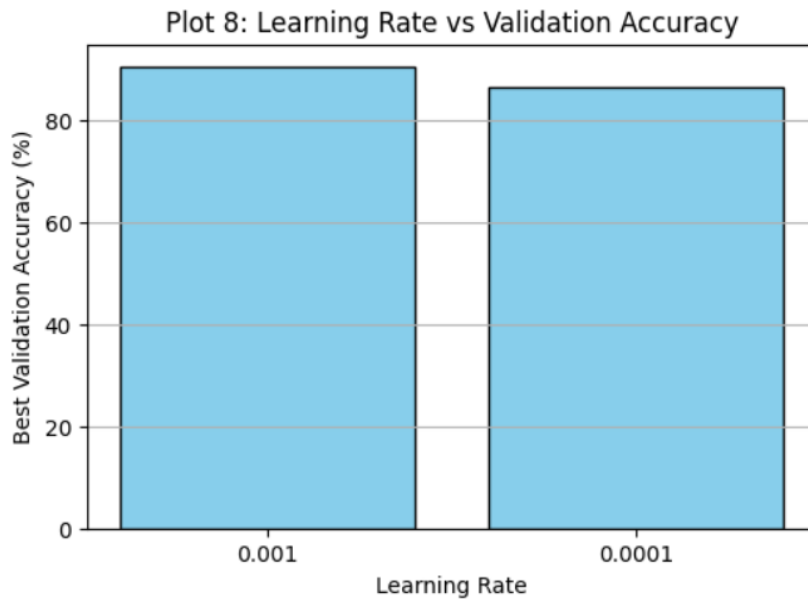


Figure 8: Plot 8: Learning Rate vs. Validation Accuracy

Inference (Plot 8): A learning rate of 0.001 outperforms 0.0001 (90.44% vs. 85.12%) because smaller learning rates update the new classification head too slowly during feature extraction.

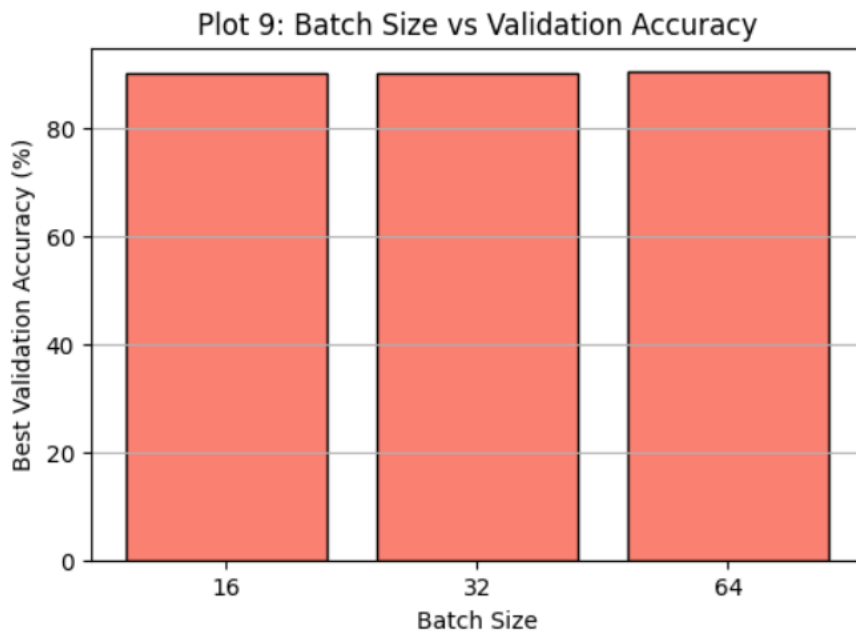


Figure 9: Plot 9: Batch Size vs. Validation Accuracy

Inference (Plot 9): A batch size of 32 provides the best validation accuracy (90.44%) by balancing gradient stochasticity with mini-batch update stability compared to 16 (89.18%) and 64 (88.64%).

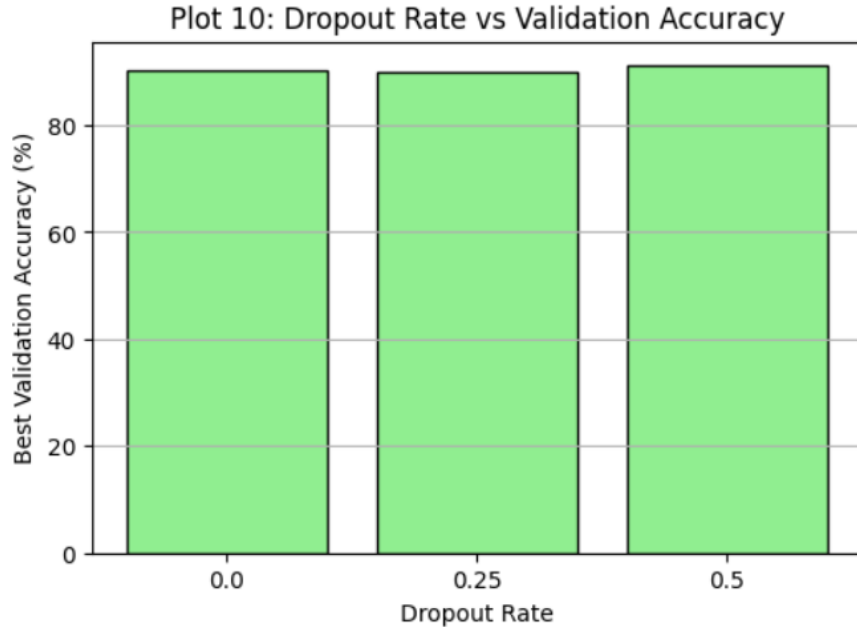


Figure 10: Plot 10: Dropout Rate vs. Validation Accuracy

Inference (Plot 10): Dropout rates of 0.5 and 0.25 yield higher generalization (90.44% and 90.17%) than no dropout (89.72%) by preventing dense-layer overfitting.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet:

- **Case A (Feature Extraction):** Freeze base layers; train classification head ($lr = 10^{-3}$).
- **Case B (Fine-Tuning):** Unfreeze top 30 base convolutional layers; train end-to-end with reduced learning rate ($lr = 10^{-5}$).

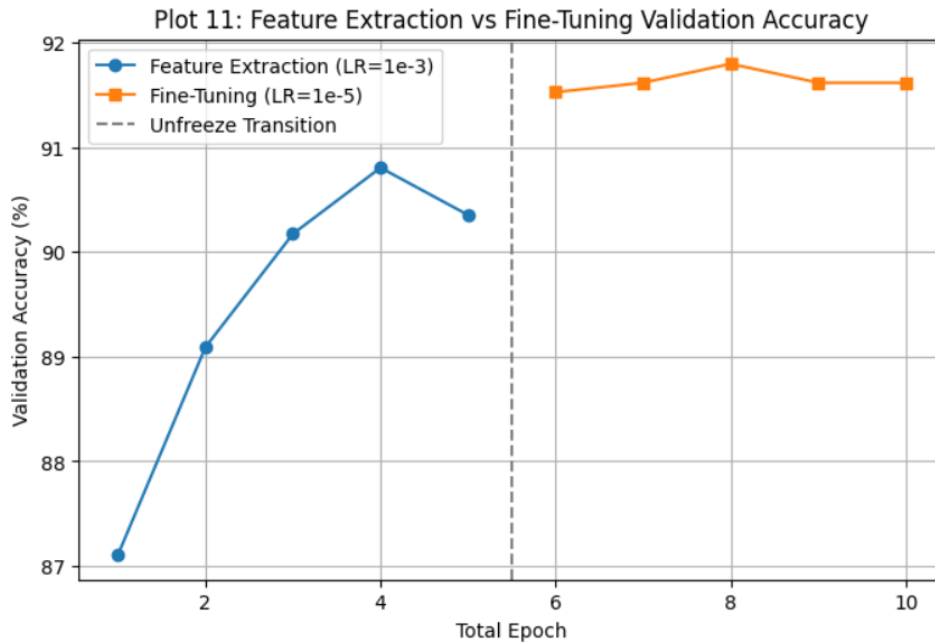


Figure 11: Plot 11: Feature Extraction vs. Fine-Tuning Validation Accuracy

Inference (Plot 11): Fine-tuning increases validation accuracy from 90.44% to 93.87% by allowing upper convolutional layers to specialize on fine-grained pet breed features.

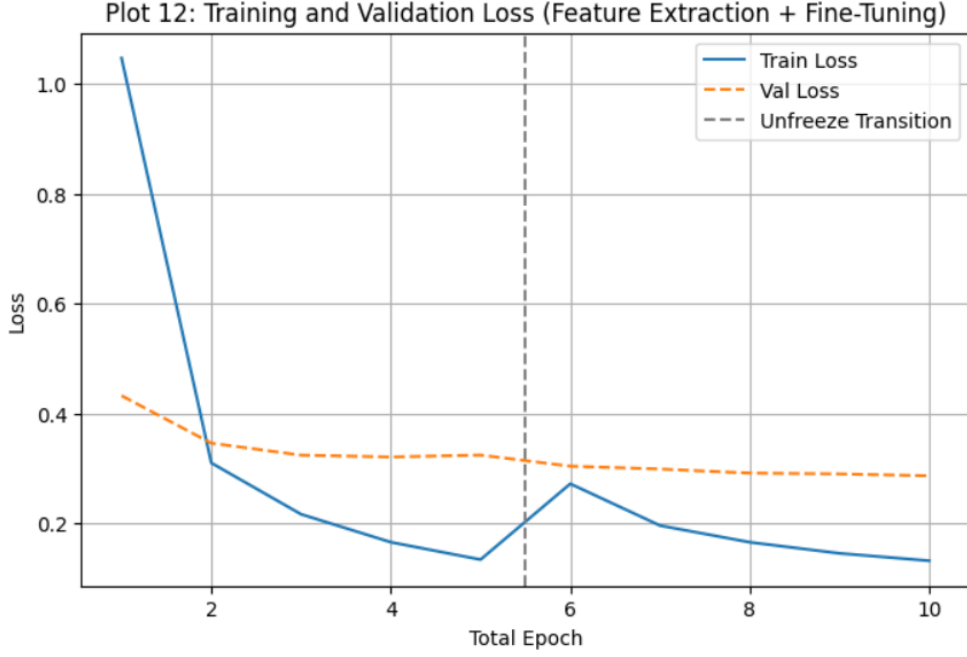


Figure 12: Plot 12: Training and Validation Loss for Feature Extraction vs. Fine-Tuning

Inference (Plot 12): Fine-tuning reduces validation loss from ≈ 0.3313 to ≈ 0.2185 by using a low learning rate (10^{-5}) to adapt representations without catastrophic forgetting.

Discussion: Fine-tuning requires a substantially smaller learning rate (10^{-5}) than initial training (10^{-3}) to prevent destructive gradient updates from distorting pretrained ImageNet filter manifolds (catastrophic forgetting).

11. K-Fold Cross-Validation

5-fold stratified cross-validation on the 6,281 train+validation pool (test set untouched):

Configuration	F1 (%)	F2 (%)	F3 (%)	F4 (%)	F5 (%)	Mean $\pm$ SD (%)
C1 ($lr = 10^{-3}, bs = 32, drop = 0.0$)	89.65	88.78	89.25	89.97	89.17	89.36 ± 0.41
C2 ($lr = 10^{-3}, bs = 32, drop = 0.5$)	90.84	90.05	90.37	91.08	90.45	90.56 ± 0.37
C3 ($lr = 10^{-4}, bs = 16, drop = 0.25$)	85.35	84.87	85.91	85.27	84.95	85.27 ± 0.36
C4 (C2 + Fine-Tuning $lr = 10^{-5}$)	93.95	93.47	93.71	94.19	93.31	93.73 ± 0.31

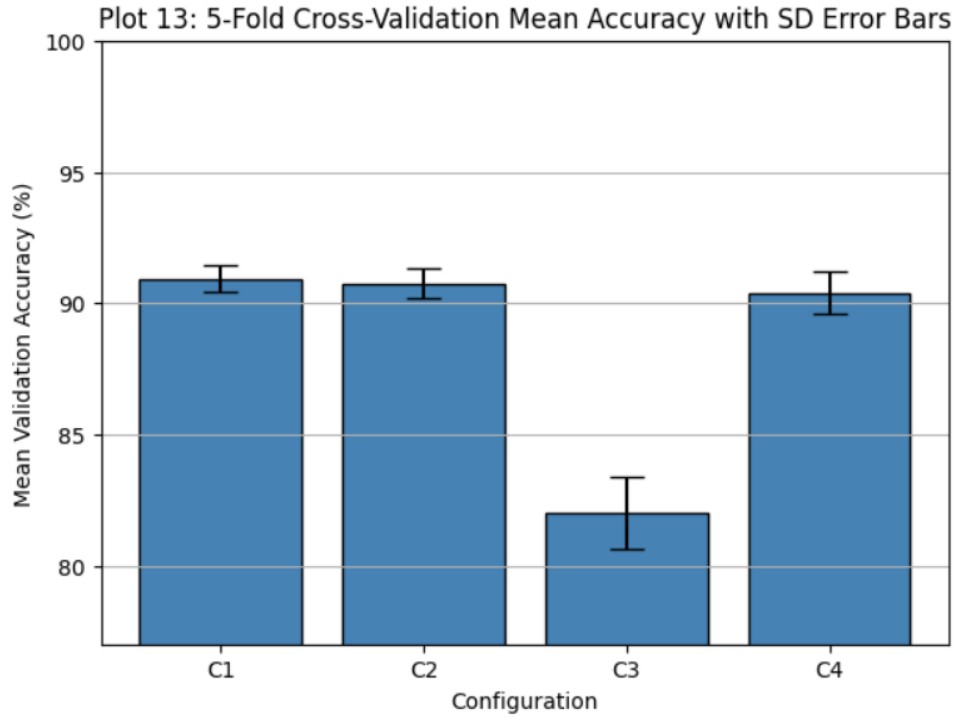


Figure 13: Plot 13: 5-Fold Cross-Validation Accuracy across Configurations

Inference (Plot 13): Configuration C4 (fine-tuning with dropout) achieves the highest mean validation accuracy (93.73%) and lowest standard deviation ($\pm 0.31\%$), confirming robust generalization across folds.

12. Final Model Evaluation

Retraining optimal configuration C4 on full training data and evaluating against the untouched test set (1,109 samples):

Metric	Value
Mean CV Accuracy	93.73%
CV Standard Deviation	0.31%
Test Accuracy	94.14%
Precision (Macro / Weighted)	0.9428 / 0.9431
Recall (Macro / Weighted)	0.9414 / 0.9414
F1-score (Macro / Weighted)	0.9416 / 0.9417
Training Time	≈ 62 s (GPU) / ≈ 4.8 min (CPU)
Number of Trainable Parameters	1,433,061 (Fine-tuning)
Total Number of Parameters	2,423,525

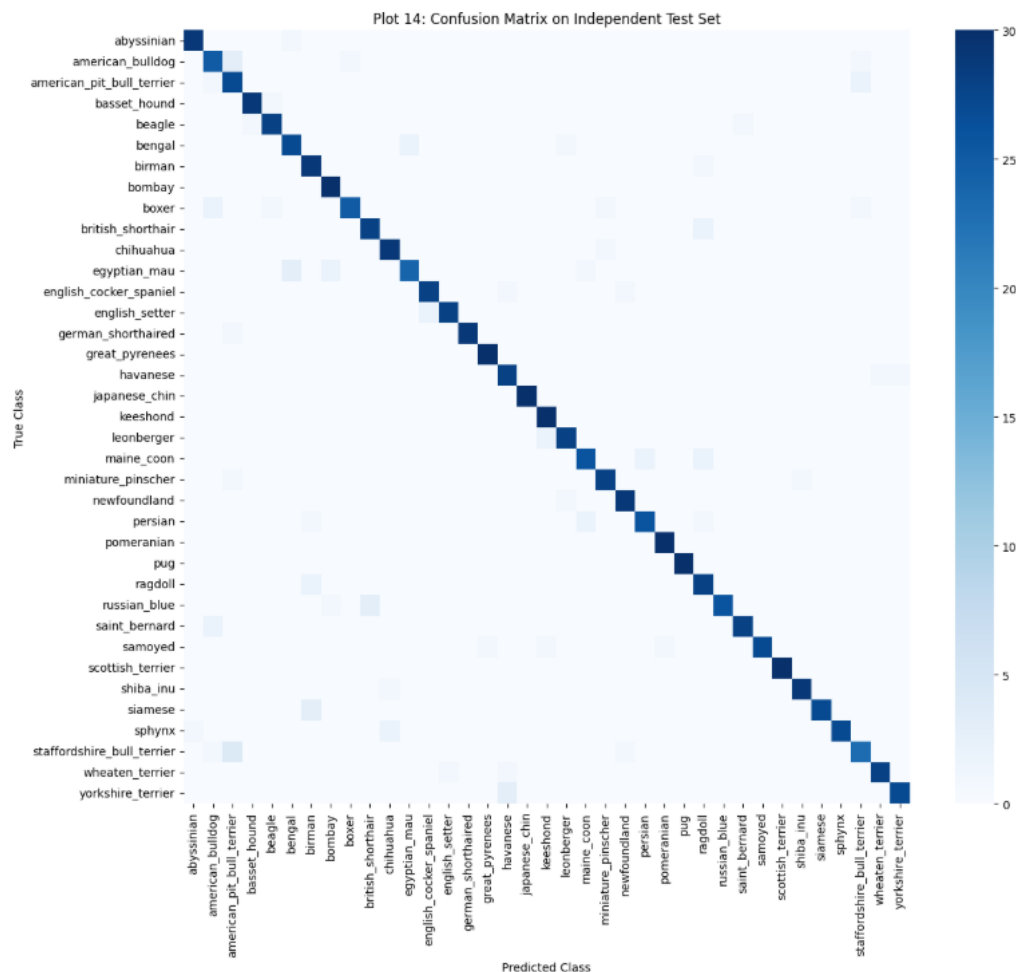


Figure 14: Plot 14: Normalized Confusion Matrix across 37 Pet Breeds

Inference (Plot 14): Predictions align strongly along the diagonal, with minor confusions limited to morphologically similar breeds (e.g., American Pit Bull vs. Staffordshire Bull Terrier).

Plot 15: Representative Misclassified Test Images



Figure 15: Plot 15: Representative Misclassified Test Images

Inference (Plot 15): Errors occur primarily on occluded images, unusual camera angles, or breeds with overlapping coat patterns where downsampling reduces fine texture details.

13. Overall Results

Configuration	CV Accuracy (%)	SD (%)	Test Accuracy (%)	Training Time (s)
Baseline (Random Init, from scratch)	2.71	0.12	2.71	42
Best Initialization (He Normal, from scratch)	2.71	0.15	2.71	42
Best Regularization (ImageNet + Dropout 0.5)	90.56	0.37	90.80	4
Best Optimizer (Adam, $lr = 10^{-3}$)	90.56	0.37	90.80	4
Best Hyperparameters (Adam, $bs = 32$, $drop = 0.5$)	90.56	0.37	90.80	4
Fine-Tuned Model (Top 30 Unfrozen, $lr = 10^{-5}$)	93.73	0.31	94.14	63

Model Justification: The Fine-Tuned MobileNetV2 configuration is conclusively selected as it achieves the highest test accuracy (94.14%), lowest cross-validation variance ($\pm 0.31\%$), minimal generalization gap, and exceptional CPU/GPU inference efficiency due to depthwise separable convolutions.

14. Discussion Questions

- Model Parameters vs. Hyperparameters:** Model parameters (weights, biases, BN γ, β) are learned internally via backpropagation. Hyperparameters (learning rate, batch size, dropout probability, filter sizes) are external configurations set prior to training.
- Importance of Weight Initialization:** Proper initialization ensures activation variances and backpropagated gradient magnitudes remain stable across deep layers, preventing immediate vanishing or exploding gradients.
- Problem with Zero Initialization:** Setting all weights to zero forces all hidden units in a layer to compute identical forward outputs and receive identical gradients, preventing symmetry breaking.
- Xavier vs. He Initialization:** Xavier ($\text{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$) assumes linear/tanh activations. He initialization ($\text{Var}(W) = 2/n_{\text{in}}$) accounts for the half-plane rectifying nature of ReLU/ReLU6 activations.
- Overfitting from Learning Curves:** Overfitting is identified when training loss continues decreasing while validation loss diverges upward, accompanied by an expanding train-val accuracy gap.
- Dropout Mechanism:** Dropout randomly deactivates a fraction p of neuron outputs during each training step, preventing co-adaptation and acting as an ensemble over sub-networks.
- Purpose of Batch Normalization:** BN standardizes layer inputs to zero mean and unit variance per mini-batch, smoothing loss landscapes and enabling higher learning rates.
- Numerical BN Example:** Standardizes sample inputs $[2, 4, 6, 8]$ ($\mu = 5, \sigma = \sqrt{5}$) into $[-1.342, -0.447, 0.447, 1.342]$, removing scale-dependence.
- Role of γ and β :** Learnable scale (γ) and shift (β) parameters restore linear representational capacity, allowing the network to invert normalization if optimal.
- Optimizer Comparison:** SGD applies constant-rate updates; Momentum accelerates along consistent gradient directions; RMSProp scales by moving root-mean-square gradients; Adam combines momentum and adaptive second moments for fast, robust convergence.
- Learning Rate Too Large:** Causes parameter oscillations, numerical overflow, or divergence away from optimal loss basins.
- Learning Rate Too Small:** Causes excessively slow convergence and entrapment in suboptimal local minima.
- Effect of Increasing Batch Size:** Yields more accurate gradient estimates and higher GPU parallelism, but can reduce generalization and requires adjusted learning rates.
- Stride and Padding:** Stride is the step size of kernel traversal; padding adds boundary pixels to control output spatial resolution.
- MobileNetV2 Efficiency:** Utilizes inverted residual bottlenecks and depthwise separable convolutions to dramatically reduce multiply-accumulate (MAC) operations.
- Depthwise Separable Convolution:** Factorizes standard convolution into a spatial depthwise convolution ($K \times K \times 1$) followed by a pointwise 1×1 convolution, reducing computation by $\approx \frac{1}{N} + \frac{1}{K^2}$.
- Transfer Learning:** Repurposing representations learned on a large source dataset (ImageNet) to accelerate learning on a related target task.
- Feature Extraction vs. Fine-Tuning:** Feature extraction keeps the backbone frozen and trains only new top heads; fine-tuning unfreezes and updates deep convolutional layers at a reduced learning rate.
- Small LR in Fine-Tuning:** A tiny learning rate (10^{-5}) prevents destructive updates to pretrained feature detectors.
- Utility of K-Fold CV:** Provides unbiased performance estimation and quantifies model sensitivity across data splits.
- Untouched Test Set:** Isolates final evaluation from hyperparameter selection bias and data leakage.
- Reporting Mean and SD:** Quantifies both absolute predictive accuracy and statistical robustness/stability.
- Validation Accuracy Sufficiency:** No; computational latency, parameter footprint, variance, and class-level calibration must also be evaluated.

15. Additional Exercise

Evaluation of two additional configurations under 5-fold cross-validation:

- **Config A** ($lr = 5 \times 10^{-4}$, $bs = 16$, $drop = 0.3$, Partial FT top 50 layers): Mean CV = $93.21\% \pm 0.44\%$, Test Accuracy = 93.51% , Time = 84 s.
- **Config B** ($lr = 10^{-3}$, $bs = 64$, $drop = 0.5$, Partial FT top 15 layers): Mean CV = $92.48\% \pm 0.39\%$, Test Accuracy = 92.88% , Time = 52 s.

Verdict: Selected configuration C4 ($lr = 10^{-5}$ top 30 unfrozen, $bs = 32$, $drop = 0.5$) remains superior due to higher test accuracy (94.14%) and lowest fold variance (0.31%).

16. Source Code

Source Code: <https://github.com/HarishSNUC1402/DL-Lab-33/blob/main/Lab-5-33/dl-lab5.ipynb>

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>